

Ejercicio 2. Investigación: qué es la Teoría de la Computación

¿Qué es la teoría de la computación?

La Teoría de la Computación es una rama de la computación y las matemáticas que se encarga de estudiar las capacidades, computacionales. Su propósito principal es responder qué problemas pueden ser resueltos mediante procedimientos algorítmicos y cuántos recursos (tiempo y memoria) se requieren para solucionarlo.

Sipser la define fundamentalmente como la búsqueda de respuestas a la pregunta: "¿Cuáles son las capacidades y limitaciones fundamentales de las computadoras?" En su enfoque, la materia busca abstraer el concepto de cómputo en modelos matemáticos para determinar qué se puede calcular y con qué costo en recursos.

Hopcroft et al. La definen como la disciplina a partir del estudio de los modelos abstractos de computadoras y los problemas que pueden (o no) resolverse mediante ellos.

La principal diferencia entre sus definiciones es que Sipser lo ve desde el punto de las capacidades y limitaciones de los computadores y Hopcroft se enfoca más en cómo los autómatas y las gramáticas formales sirven como base para definir sistemas de procesamiento de información y lenguajes.

Orígenes de la Teoría de la Computación

El origen de la Teoría de la Computación nació de una crisis conceptual en las matemáticas a principios del siglo XX. El intento de fundamentar toda la matemática mediante la lógica llevó al descubrimiento de paradojas, lo que derivó en la formulación del Programa de Hilbert y el famoso Entscheidungsproblem.

A finales del siglo XIX, matemáticos como Gottlob Frege y Georg Cantor intentaron construir una base rigurosa para las matemáticas utilizando la teoría de conjuntos y la lógica formal. Sin embargo, aparecieron contradicciones internas conocidas como paradojas como:

La Paradoja de Russell (1901): Bertrand Russell propuso el conjunto R de todos los conjuntos que no se contienen a sí mismos como miembros

$$R = \{x \mid x \text{ pertenece a } x\}$$

- si nos preguntamos si R pertenece a R
- R por definición no debería de contenerse a sí mismo
- pero si R no pertenece a $R \Rightarrow R \text{ pertenece a } R$

Esta contradicción demostró que la teoría intuitiva de conjuntos de Frege era inconsistente. De la misma manera la Paradoja de Richard y la Paradoja de Burali-Forti mostraron contradicciones similares al definir construcciones autorreferenciales con lenguaje matemático y ordinales. Estas contradicciones provocaron la "crisis de los fundamentos de las matemáticas", ya que un sistema lógico que contenga una sola contradicción permite demostrar cualquier enunciado (principio de explosión), volviendo inútil a toda la disciplina.

Para resolver esta crisis y volver a creer en las matemáticas de una base inquebrantable, el matemático alemán David Hilbert propuso en la década de 1920 un ambicioso proyecto conocido como el Programa de Hilbert.

Hilbert planteó formalizar todas las matemáticas en un lenguaje axiomático estricto que cumpliera tres propiedades fundamentales:

1. Consistencia: Demostrar rigurosamente que el sistema no contiene contradicciones (es decir, que no se puede probar P y $\neg P$ a la vez).

2. **Compleitud:** Demostrar que toda proposición matemática verdadera expresada en el sistema puede ser probada dentro del mismo sistema.

3. **Decidibilidad (Entscheidungsproblem):** Encontrar un procedimiento mecánico y finito para determinar la verdad o falsedad de cualquier enunciado.

Hilbert confiaba plenamente en que la respuesta a estas exigencias sería afirmativa, bajo su célebre lema: "Wir müssen wissen, wir werden wissen" ("Debemos saber, sabremos").

El Entscheidungsproblem (El problema de la decisión)

Este fue formulado por Hilbert y Wilhelm Ackermann en 1928, el Entscheidungsproblem hace la pregunta de ¿Existe un procedimiento mecánico paso a paso que, dada una frase en la lógica de primer orden, determine en un número finito de pasos si dicha frase es válidamente deducible a partir de los axiomas?

Para responder a esta pregunta, los matemáticos primero necesitaban definir formalmente qué es un "procedimiento mecánico" lo que hoy conocemos como un algoritmo pero cuyo concepto no existía formalmente en ese entonces.

La búsqueda de la respuesta al Entscheidungsproblem dismanteló el Programa de Hilbert, pero dio nacimiento a la computación teórica a través de dos hitos principales:

Los Teoremas de Incompletitud de Gödel (1931)

Kurt Gödel destruyó los dos primeros enunciados de Hilbert al demostrar que:

- Cualquier sistema axiomático formal lo suficientemente potente como para describir la aritmética básica es incompleto (existen enunciados verdaderos que no se pueden probar dentro del sistema).
- La consistencia del sistema no se puede probar dentro del propio sistema.

La respuesta al Entscheidungsproblem: Turing y Church (1936)

Para resolver el tercer enunciado de Hilbert (la decidibilidad), se necesitaba formalizar la noción de algoritmo:

Alan Turing (1936): Formuló un modelo teórico de máquina con una cinta infinita y un estado de control: la Máquina de Turing. Con ella, formalizó la noción intuitiva de "cómputo" y demostró que el Entscheidungsproblem tiene una respuesta negativa al probar que el Problema de la Parada (Halting Problem) es indecidible.

Alonzo Church (1936): Desarrolló de forma independiente el Cálculo Lambda y llegó a la misma conclusión de indecidibilidad.

La equivalencia entre la Máquina de Turing y el Cálculo Lambda dio lugar a la Tesis de Church-Turing, la cual establece que cualquier proceso computable mediante un algoritmo puede ser ejecutado por una Máquina de Turing.

La Tesis de Church-Turing es la afirmación fundamental que establece la equivalencia entre la noción intuitiva e informal de un "algoritmo" y la noción matemática formal definida mediante modelos de cómputo abstractos. La tesis postula que cualquier función o problema que un ser humano o una máquina pueda calcular mediante un procedimiento sistemático, finito y mecánico (un algoritmo), puede ser calculado por una Máquina de Turing (o cualquier modelo formal equivalente).

Y a esta se le conoce como tesis y no teorema porque en matemáticas, un teorema es un enunciado que se demuestra formalmente mediante deducción lógica a partir de un conjunto de axiomas prefijados.

Sin embargo, la Tesis de Church-Turing no se puede decir como un teorema formalmente es porque es imposible de demostrar porque conceta un concepto informal y muy abstracto que es algoritmo con un concepto matematico muy complejo que es la amquina de turing y el teorema lambda. Como no se puede dar una prueba formal sobre un concepto que es informal la afirmacion permanece como una tesis pero este es aceptado mundialmente debido a que no se ha encontrado forma de desmentirlo y

Ramas de la teoria de la computacion

La Teoría de la Computación se divide estructuralmente en tres grandes ramas. Cada una aborda el fenómeno del cómputo desde un ángulo distinto y responde a una pregunta fundamental.

1. Teoría de Autómatas y Lenguajes Formales=>Estudia los modelos matemáticos abstractos de cómputo (autómatas) y la estructura de la información que estos pueden procesar (lenguajes formales y gramáticas). Clasifica las máquinas según su capacidad de memoria y reglas de transición.

Responde a la pregunta **¿Qué tipos de modelos de cómputo existen y qué clase de lenguajes o patrones son capaces de reconocer y procesar?**

2. Teoría de la Computabilidad=>Analiza los límites absolutos de la computación, independientemente de la potencia del hardware o del tiempo disponible. Clasifica los problemas en decidibles (se pueden resolver con un algoritmo) e indecidibles (es imposible construir un algoritmo que los resuelva para todos los casos).

Responde a la pregunta **¿Existen problemas bien definidos que NINGUNA computadora, sin importar el tiempo ni la memoria, pueda resolver jamás?**

3. Teoría de la Complejidad Computacional=>Clasifica los problemas computables según los recursos requeridos para resolverlos (principalmente tiempo de ejecución y espacio en memoria). Separa los problemas que son tratables de forma práctica en el mundo real de aquellos que son intrínsecamente inabordables a medida que crece el tamaño de la entrada.

Responde a la pregunta **¿Qué hace que un problema sea intrínsecamente "fácil" o "difícil" de resolver, y cuántos recursos (tiempo y memoria) requiere su solución?**

Conceptos Fundamentales

Alfabeto (Σ)

Un **alfabeto** es un conjunto finito no vacío de símbolos abstractos. Se denota por la letra griega mayúscula Σ

Ejemplo= $\sum_1^+ \{a, b, c, d\}$, $\sum_2^+ \{1, 2, 3, 4\}$

Cadena (Palabra o caracteres)

Una **cadena** sobre un alfabeto Σ es una secuencia finita de símbolos elegidos de Σ .

Longitud de una cadena $|w|$: El número de símbolos que forman la cadena w .

Cadena vacía (λ): Es la secuencia que contiene cero símbolos. Su longitud es $|\lambda|=0$.

Lenguaje (L): Un lenguaje sobre un alfabeto Σ es un conjunto de cadenas formadas por símbolos de Σ
Formalmente, L es un lenguaje si $L \subseteq \sum_{\Sigma}^+$

Ejemplos

$L_1 = \{ab, aabb, aaabbb\}$ (lenguaje finito).

$L_2 = \{a^n b^n \mid n \geq 0\}$ (lenguaje infinito).

$L_3 = \{\emptyset\}$ (el lenguaje vacío, que no contiene cadenas).

$L_4 = \{\lambda\}$ (el lenguaje que contiene únicamente la cadena vacía).

Operaciones sobre Cadenas

Sean $u = a_1 a_2 \dots a_n$ & $v = b_1 b_2 \dots b_m$ dos cadenas sobre Σ

1. Concatenación ($u \cdot v$ o uv)

Definición: $u \cdot v = u_1 u_2 \dots u_n v_1 v_2 \dots v_m$

Propiedades: $|uv| = |u| + |v|$ y el elemento neutro es λ donde $(\lambda u = u\lambda = u)$.

2. Potencia (w^n)

Representa la concatenación repetida de una cadena w consigo misma n veces.

Definición $w^n = w \cdot w^{(n-1)}$ para $n \geq 1$

Y para $w^0 = \lambda$

3. Reflexión o Inversión (w^R)

Es la cadena formada por los mismos símbolos de w , pero en orden inverso.

$(ua)^R = a^R u^R$ donde u pertenece Σ^* y a pertenece Σ

$\lambda^R = \lambda$

Ejemplo:

$w = abc \Rightarrow w^R = cba$

4. Unión ($L_1 \cup L_2$)

$L_1 \cup L_2 = \{w \mid w \text{ pertenece } L_1 \vee w \text{ pertenece } L_2\}$

5. Intersección ($L_1 \cap L_2$)

$L_1 \cap L_2 = \{w \mid w \text{ pertenece } L_1 \wedge w \text{ pertenece } L_2\}$

6. Diferencia ($L_1 \setminus L_2$)

$L_1 \setminus L_2 = \{w \mid w \text{ pertenece } L_1 \wedge w \text{ no pertenece } L_2\}$

7. Cerradura de Kleene (L^*)

Es la unión infinita de todas las potencias de L , partiendo desde 0.

$L^* = \bigcup_{i=0}^{\infty} L^i = L^0 \cup L^1 \cup L^2 \cup \dots$

8. Cerradura Positiva (L^+)

Es la unión infinita de todas las potencias de L , partiendo desde 1.

$L^+ = \bigcup_{i=1}^{\infty} L^i = L^1 \cup L^2 \cup L^3 \cup \dots$

¿Por qué $\Sigma^0 = \{\lambda\}$?

Por definición de la operación de potencia de un conjunto/lenguaje, Σ^n representa el conjunto de todas las cadenas de longitud exactamente igual a n formadas con los símbolos del alfabeto Σ

Entonces $\Rightarrow \Sigma^0 = \{\text{una cadena con magnitud } 0 \text{ que es } \lambda\}$

¿Qué distingue a Σ^* de Σ^+ ?

Que la cerradura de Kleene si considera la cadena vacía λ y la cerradura positiva no la considera en el alfabeto.

La Jerarquía de Chomsky

Fue propuesta por Noam Chomsky en 1956) clasifica las gramáticas formales, los lenguajes que estas generan y las máquinas (autómatas) capaces de reconocerlos en cuatro niveles estructurados de manera inclusiva: todo lenguaje de Tipo n es también de Tipo $n-1$.

Tipo	Lenguaje Generado	Gramática Correspondiente	Formato de las Reglas de Producción ($\alpha \Rightarrow \beta$)	Máquina / Autómata Reconocedor
Tipo 0	Lenguajes Sin Restricciones (<i>Recursivamente Enumerables</i>)	Gramática Sin Restricciones (Unrestricted)	$\alpha \Rightarrow \beta$ Donde α pertenece $(V \cup T)^{+}$ conteniendo al menos un no terminal, y β pertenece $(V \cup T)^{*}$. Sin restricciones.	Máquina de Turing (MT)
Tipo 1	Lenguajes Sensibles al Contexto (<i>Context-Sensitive</i>)	Gramática Sensible al Contexto (CSG)	$AA\beta \Rightarrow \alpha\tau\beta$ Donde A es no terminal α, β, τ pertenece $(V \cup T)^{*}$ con τ diferente λ o alternativamente $ \alpha \leq \beta $	Autómata con Límite Lineal (LBA / Linear Bounded Automaton)
Tipo 2	Lenguajes Libres del Contexto (<i>Context-Free</i>)	Gramática Libre del Contexto (CFG)	$A \Rightarrow \tau$ Donde A es el único símbolo no terminal (A pertenece V) Y τ pertenece $(V \cup T)^{*}$	Autómata de Pila (PDA / Pushdown Automaton)
Tipo 3	Lenguajes Regulares (<i>Regular</i>)	Gramática Regular (<i>Lineal Derecha o Lineal Izquierda</i>)	$A \Rightarrow aB$ o $A \Rightarrow a$ Donde A, B pertenece V y a pertenece T pudiendo incluir la λ	Autómata Finito (<i>DFA / NFA</i>)

Leyenda: V = Variables o No Terminales; T = Terminales; λ = Cadena vacía.

Definición Formal de AFD y AFN

Un autómata finito es un modelo matemático de cómputo con memoria finita que cambia de estado según los símbolos de entrada recibido

AFD=> En un AFD, para cada estado y cada símbolo del alfabeto existe **exactamente una única transición** hacia un estado siguiente.

Formalmente, un AFD es una 5-tupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

1. **Q**: Conjunto finito y no vacío de **estados**.
2. **Σ : Alfabeto** de entrada (conjunto finito de símbolos).
3. **δ : Función de transición**, definida como $\delta: Q \times \Sigma \Rightarrow Q$
4. q_0 : **Estado inicial** donde q_0 pertenece Q
5. F : Conjunto de estado finales o de aceptación, donde $F \subseteq Q$

AFN=> En un AFN, para un estado y un símbolo de entrada, pueden existir **cero, una o múltiples transiciones** a distintos estados. Además, se permiten transiciones con la cadena vacía λ conocidas como λ -transiciones, que permiten cambiar de estado sin consumir símbolos de la entrada.

Formalmente, un AFN en una 5-tupla.

$$M = (Q, \Sigma, \delta, q_0, F)$$

Tiene el mismo significado que el AFN

Lo único que cambia es:

$$\delta: Q \times (\Sigma \cup \{\lambda\}) \Rightarrow P(Q)$$

donde $P(Q)$ es el conjunto potencia de Q , es decir el conjunto de todos los subconjuntos posibles de estados.

Equivalencia entre AFD y AFN

ambos modelos tienen exactamente el mismo poder de cómputo: reconocen exactamente la misma clase de lenguajes (los **Lenguajes Regulares**, Tipo 3 de la Jerarquía of Chomsky). Aunque el AFN parece un poco mas flexible que AFD

Problemas Reales donde se Aplican Expresiones Regulares

1. Validación de Sintaxis y Formatos de Entrada

En formularios web o bases de datos, es necesario garantizar que los datos ingresados por un usuario sigan una estructura estricta antes de procesarlos. La solución son validaciones de correos electrónicos, números telefónicos, códigos postales, contraseñas seguras o direcciones IP.

2. Análisis Léxico (Lexing / Scanning) en Compiladores e Intérpretes

Un compilador recibe un archivo de código fuente (una cadena gigante de caracteres) y debe identificar las unidades con significado (tokens: palabras clave, identificadores, operadores, literales). Se soluciona con herramientas como flex o lex utilizan expresiones regulares para definir patrones léxicos y generar un autómata finito determinista que escanea el código a velocidad constante $O(n)$.

3. Búsqueda y Extracción de Patrones en Grandes Volúmenes de Texto

Al Analizar archivos de registros (logs) de servidores web para detectar amenazas de seguridad (ej. inyecciones SQL), extracción de información (web scraping) o comandos de consola como grep y sed. Se soluciona buscando patrones de ataques, filtrar líneas específicas dentro de millones de datos o realizar sustituciones masivas en archivos de texto.

Referencias:

- **Michael Sipser:** *Introduction to the Theory of Computation* (3.^a edición, Cengage Learning, 2012).
- Computerphile. (2014, 18 de marzo). *Turing Machines explained - Computerphile* [Archivo de Video]. YouTube. <https://www.youtube.com/watch?v=dNRDvLACg5Q>
- Peñaloza, G. [Gabriel Peñaloza]. (2020). *Curso de teoría de la computación y autómatas* [Lista de reproducción de YouTube]. YouTube. https://www.youtube.com/playlist?list=PL_X3
- **John E. Hopcroft, Rajeev Motwani y Jeffrey D. Ullman:** *Introduction to Automata Theory, Languages, and Computation* (3.^a edición, Addison-Wesley, 2006).
- **Paradoja de Russell y Crisis de Fundamentos:** Russell, B. (1902), *Letter to Frege*, y la formalización en Whitehead, A. N., & Russell, B. (1910), *Principia Mathematica*.
- Easy Theory. (2020). *Theory of Computation / Automata Theory* [Lista de reproducción de YouTube]. YouTube. <https://www.youtube.com/c/EasyTheory>
- **Programa de Hilbert y Entscheidungsproblem:** Hilbert, D., & Ackermann, W. (1928), *Grundzüge der theoretischen Logik*.
- CdeCiencia. (2018, 14 de mayo). *El problema que la ciencia NO puede resolver* [Archivo de Video]. YouTube. <https://www.youtube.com/watch?v=R9S3R8DAn4M>
- QuantumFracture. (2019, 10 de diciembre). *¿Por qué NO podemos saberlo TODO? El Problema de la Parada* [Archivo de Video]. YouTube. <https://www.youtube.com/watch?v=92WHN-pAFCs>
- Veritasium en Español. (2021, 23 de abril). *El Teorema de Incompletitud de Gödel* [Archivo de Video]. YouTube. <https://www.youtube.com/watch?v=O4ndIDcDSSc>